

Report conversation

I want to refactor this program, which takes in an amount of change as a user input, and then outputs how many of each bill and coin you would give back, starting from largest possible bill to pennies. The areas that I believe can be optimized would be assembling all of the get change functions to a single function, as well as an error check for if the user doesn't input a proper float value:

```
import sys
```

```
def get_twenties(amount):  
    twentyBills = 0  
    while True:  
        if amount < 20:  
            break  
        amount -= 20  
        twentyBills += 1  
    return twentyBills
```

```
def get_tens(amount):  
    bills = 0  
    while True:  
        if amount < 10:  
            break  
        amount -= 10  
        bills += 1  
    return bills
```

```
def get_fives(amount):  
    bills = 0  
    while True:  
        if amount < 5:  
            break  
        amount -= 5  
        bills += 1  
    return bills
```

```
def get_ones(amount):
```

```
    bills = 0
```

```
    while True:
```

```
        if amount < 1:
```

```
            break
```

```
        amount -= 1
```

```
        bills += 1
```

```
    return bills
```

```
def get_quarters(amount):
```

```
    coins = 0
```

```
    while True:
```

```
        if amount < 0.25:
```

```
            break
```

```
        amount -= 0.25
```

```
        coins += 1
```

```
    return coins
```

```
def get_dimes(amount):
```

```
    coins = 0
```

```
    while True:
```

```
        if amount < 0.10:
```

```
            break
```

```
        amount -= 0.10
```

```
        coins += 1
```

```
    return coins
```

```
def get_nickels(amount):
```

```
    coins = 0
```

```
    while True:
```

```
        if amount < 0.05:
```

```
            break
```

```
        amount -= 0.05
```

```
        coins += 1
```

```
    return coins
```

```
def get_pennies(amount):
```

```
    coins = 0
```

```
    while True:
```

```
        if amount < 0.01:
```

```
        break
    amount -= 0.01
    coins += 1
    return coins

if __name__ == '__main__':
    changeStr = input("Provide an amount in change
(<=$100): $")
    change = float(changeStr)

    if change > 100:
        print("Error: Amount too large")
        sys.exit()

    twenties = get_twenties(change)
    change = change - (twenties * 20)

    tens = get_tens(change)
    change = change - (tens * 10)

    fives = get_fives(change)
    change = change - (fives * 5)

    ones = get_ones(change)
    change = change - ones

    quarters = get_quarters(change)
    change = change - (quarters * 0.25)

    dimes = get_dimes(change)
    change = change - (dimes * 0.10)

    nickels = get_nickels(change)
    change = change - (nickels * 0.05)

    pennies = get_pennies(change)
    change = change - (pennies * 0.01)

    print("Change to give back:")
    print("\tTwenties($20): " + str(twenties))
```

```

print("\tTens($10): " + str(tens))
print("\tFives($5): " + str(fives))
print("\tOnes($1): " + str(ones))
print("\tQuarters(25¢): " + str(quarters))
print("\tDimes(10¢): " + str(dimes))
print("\tNickels(5¢): " + str(nickels))
print("\tPennies(1¢): " + str(pennies))

```

You can optimize the code by consolidating the bill and coin calculations into a single function, which will make the code cleaner and more maintainable. Additionally, you can add error handling to ensure that the user inputs a valid floating-point number. Here's the refactored version:

python

 Copy code

```

import sys

def get_change(amount, denomination):
    count = 0
    if denomination >= 1:
        while amount >= denomination:
            amount -= denomination
            count += 1
    else:
        while amount >= denomination:
            amount -= denomination
            count += 1
    return count, amount

if __name__ == '__main__':
    try:
        changeStr = input("Provide an amount in change (<= $100): $")
        change = float(changeStr)
    except ValueError:
        print("Error: Please provide a valid numerical input.")
        sys.exit()

    if change > 100:
        print("Error: Amount too large")
        sys.exit()

```

```

denominations = [
    (20, "Twenties($20)"),
    (10, "Tens($10)"),
    (5, "Fives($5)"),
    (1, "Ones($1)"),
    (0.25, "Quarters(25¢)"),
    (0.10, "Dimes(10¢)"),
    (0.05, "Nickels(5¢)"),
    (0.01, "Pennies(1¢)")
]

print("Change to give back:")
for denomination, name in denominations:
    count, change = get_change(change, denomination)
    print(f"\t{name}: {count}")

```

This code will handle user input errors gracefully and consolidate the logic for calculating change into a single function. Additionally, it iterates over a list of denominations, which makes it easier to maintain and modify in the future.

could also add an error check if the inputted float value only has up to decimal places (a proper money format)?

Certainly! You can add a check to ensure that the inputted float value has up to two decimal places, which is a common representation for currency. Here's how you can modify the code to include this check:

python

 Copy code

```

import sys

def get_change(amount, denomination):
    count = 0
    if denomination >= 1:
        while amount >= denomination:
            amount -= denomination
            count += 1

```

```
    else:
        while amount >= denomination:
            amount -= denomination
            count += 1
        return count, amount

if __name__ == '__main__':
    try:
        changeStr = input("Provide an amount in change (<= $100): $")
        change = float(changeStr)
    except ValueError:
        print("Error: Please provide a valid numerical input.")
        sys.exit()

    if change > 100:
        print("Error: Amount too large")
        sys.exit()

    # Check if the inputted float value has up to two decimal places
    if not changeStr.replace('.', '', 1).isdigit():
        print("Error: Please provide a valid monetary format.")
        sys.exit()

    denominations = [
        (20, "Twenties($20)"),
        (10, "Tens($10)"),
        (5, "Fives($5)"),
        (1, "Ones($1)"),
        (0.25, "Quarters(25¢)"),
        (0.10, "Dimes(10¢)"),
        (0.05, "Nickels(5¢)"),
        (0.01, "Pennies(1¢)")
    ]

    print("Change to give back:")
    for denomination, name in denominations:
        count, change = get_change(change, denomination)
        print(f"\t{name}: {count}")
```

This addition checks if the string representation of the float value, after removing the decimal point, consists of only digits. If it does not, it means the float value does not have up to two decimal places, and thus, it's not in proper monetary format.